



LAB TUNINGS RUNBOOK

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

Nested vSAN ESXi Tunings for VCF 9 Lab Deployments

The five host advanced settings William Lam documents — what each does, why they matter for VCFA bootstrap, the evidence from our 2026-05-19 LCMVSPHERECONFIG1000095 failure, and the verified apply procedure.

NESTED VSAN

VCFA

LCMVSPHERECONFIG1000095

ETCD

WILLIAMLAM

VCF 9.0.1

VMware Cloud Foundation

Nested vSAN ESXi Tunings for VCF 9 Lab Deployments

Prepared by: Virtual Control LLC **Date:** May 19, 2026 **Document Version:** 1.0 **Classification:** Internal — Lab Environment **VCF Version:** 9.0.1.0 **Source:** William Lam — "VCF 9.1 Comprehensive ESX Configuration Workarounds for Lab Deployments" (2026-05) **Cluster:** vcenter-cl01 — 4-host nested ESXi on Dell Precision 7920

Scope. This runbook documents the five ESXi advanced settings William Lam identifies as required for VCF 9 lab deployments on nested vSAN. It explains what each setting does, the failure mode you hit when they are wrong, the evidence from today's LCMVSPHERECONFIG1000095 etcd failed: reason withheld incident on env c d696d6b , and the apply + verify procedure that was executed on all 4 hosts at 12:18 UTC on 2026-05-19.

Companion docs. - [Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18.md](#) — the cleanup procedure for the failed env that triggered this investigation - [Nested-ESXi-RAM-Rebalance-for-VCFA-Deploy-2026-05-18.md](#) — host-level memory work done immediately before this finding - [VCF-Automation-Deployment-Troubleshooting-Report.md](#) Section 11.5 — the April 2026 etcd-on-HDD RCA that pointed at storage as the failure axis

Table of Contents

1. TL;DR
2. The Five Settings
3. Why Each Setting Matters on Nested vSAN
4. The Failure Mode When They Are Wrong
5. The 2026-05-19 Incident — Evidence
6. Apply Procedure (Verified)
7. Verification
8. When NOT to Apply These
9. Incident Log — 2026-05-19
10. Lessons Learned
11. Document Information

1. TL;DR

If you are running VCF 9 on a nested ESXi lab where the nested ESXi VMs participate in their own inner vSAN cluster, set these **five advanced settings on every nested ESXi host** before deploying VCFA. The default ESXi values are designed for physical hardware and will cause the VCFA bootstrap VM's internal etcd to fail with LCMVSPHERECONFIG1000095 "Failed to create services platform cluster."

```
esxcli system settings advanced set -o /LSOM/VSANDeviceMonitoring -i 0
esxcli system settings advanced set -o /LSOM/lomSlowDeviceUnmount -i 0
```

```
esxcli system settings advanced set -o /VSAN/SwapThickProvisionDisabled -i 1
esxcli system settings advanced set -o /VSAN/GuestUnmap -i 1
esxcli system settings advanced set -o /VSAN/FakeSCSIReservations -i 1
```

These are runtime-tunable. **No host reboot needed.** Settings take effect on subsequent vSAN operations.

Per William Lam: "If you use the Nested ESX Virtual Appliance Fling, the vSAN optimization settings are not required as these are already applied by default." We deployed our nested ESXi via the regular installer + manual config, so the defaults are still in place and the tunings have to be applied explicitly.

2. The Five Settings

#	Setting	Default	VCF 9 Lab Target	What It Controls
1	/LSOM/VSANDeviceMonitoring	1	0	LSOM (Log Structured Object Manager) actively monitors physical device health/latency and can mark devices as unhealthy
2	/LSOM/lsomSlowDeviceUnmount	1	0	LSOM auto-unmounts devices it perceives as "slow" (latency above thresholds)
3	/VSAN/SwapThickProvisionDisabled	1	1	When 0, swap files are thick-provisioned; when 1, thin (saves capacity in lab)
4	/VSAN/GuestUnmap	0	1	Pass TRIM/UNMAP from guest VMs through to vSAN so freed space gets reclaimed
5	/VSAN/FakeSCSIReservations	0	1	Allows fake SCSI reservation handling needed for clustered vSphere features when the nested layer can't pass through real SCSI reservations

The default on this homelab (2026-05-19, pre-fix): 1, 1, 1, 0, 0 — three of the five wrong.

Verified post-fix on all 4 hosts (2026-05-19 12:18 UTC): 0, 0, 1, 1, 1 ✓

3. Why Each Setting Matters on Nested vSAN

3.1 /LSOM/VSANDeviceMonitoring = 0

On physical hardware, LSOM's device monitor watches SMART data and physical I/O latency to detect failing drives. On nested vSAN, the "devices" LSOM sees are actually virtual disks backed by a physical ESXi host below. The latency LSOM observes is dominated by the layer below's own scheduling, vSAN write amplification, and the outer host's own load — not by the underlying physical media.

With monitoring on (default): LSOM will see "abnormal" latency that is actually normal for nested operation, log warnings, and potentially trigger device health responses that don't match the real hardware state.

With monitoring off: LSOM trusts the inner virtual disks at face value and doesn't generate spurious health events.

3.2 `/LSOM/lSomSlowDeviceUnmount = 0`

This is the **most operationally dangerous default** for nested vSAN. With it enabled, LSOM can decide that a virtual device is too slow and **unmount it from the vSAN disk group mid-operation**. The disk group becomes degraded. Objects on it become inaccessible. The bootstrap VM's etcd, which was writing to that disk group, loses its backing store.

With unmount on (default): under burst write load — exactly what VCFA bootstrap creates — LSOM can pull a virtual disk out from under a running guest. Recovery requires manually re-adding the disk group.

With unmount off: LSOM keeps the disk mounted regardless of perceived latency. The inner VM's I/O slows down but the storage stays consistent. The bootstrap completes; etcd survives.

3.3 `/VSAN/SwapThickProvisionDisabled = 1`

Thin-provisioned swap files. Default is already correct for lab use. Saves capacity since most VMs never use their swap.

3.4 `/VSAN/GuestUnmap = 1`

Without this, when a guest VM deletes data and issues TRIM/UNMAP, the freed blocks are not reclaimed by vSAN. Capacity slowly fills with "dead" data. On a lab cluster with limited capacity headroom this accelerates the time-to-full and amplifies write performance issues (full disks are slower).

With UNMAP on: guest deletes free up space at the vSAN layer too.

3.5 `/VSAN/FakeSCSIReservations = 1`

vSphere features like MSCS (Microsoft Cluster Service) and some K8s storage drivers issue SCSI reservation operations. On nested vSAN, the inner layer can't always pass these reservations to the underlying physical layer. Without this setting, those operations fail. With it, vSAN reports successful reservation to the guest while handling consistency internally.

For VCFA: K8s components like `vsphere-csi-node` issue reservation-style operations. The bootstrap log from the failed deploy shows `vsphere-csi-node-jmj45` having liveness probe failures — partially related.

4. The Failure Mode When They Are Wrong

The mechanism is a **cascading control plane failure inside the VCFA bootstrap VM**:

1. Bootstrap VM `vcfa-mgmt-*` boots on a nested ESXi host
2. Inside the bootstrap VM, kubeadm initializes a local Kubernetes cluster
3. etcd starts — it writes to a virtual disk backed by nested vSAN
4. etcd's defaults: `election-timeout: 10000ms`, `heartbeat-interval: 500ms`
5. The bootstrap workflow proceeds to deploy the VCFA registry pod, generating burst writes
6. **Under burst writes, nested vSAN's effective fsync latency spikes** — especially with LSOM monitoring on, because LSOM throttles I/O to "slow" devices
7. **If `lSomSlowDeviceUnmount=1`**, the disk group can briefly unmount
8. etcd's heartbeat interval (500 ms) gets missed
9. After 5 missed heartbeats, etcd considers the member offline
10. etcd attempts re-election — but the election requires fsync, which also takes longer than the 10-second election-timeout

11. Election fails → etcd enters CrashLoopBackOff
12. kube-apiserver's `etcd` health check returns `[-]etcd failed: reason withheld`
13. kube-apiserver becomes unhealthy
14. All controllers (kube-controller-manager, kube-scheduler) lose their leader leases — exactly the same lease-loss pattern we saw in Fleet's KIND cluster yesterday, but inside the bootstrap VM this time
15. Pods like `trust-manager`, `capv-controller-manager`, `logging-operator` all fail to acquire leases — log shows `connection refused to the cluster service IP 198.19.0.1:443`
16. The bootstrap workflow's `Deploy registry on VCF services platform cluster` step exceeds its timeout
17. Fleet declares the deploy `FAILED` with error code `LCMVSPHERECONFIG1000095`

5. The 2026-05-19 Incident — Evidence

5.1 Deploy Timeline (env `cd696d6b`)

Time (UTC)	Event
04:25:41	API POST to <code>/lcm/lcops/api/v2/environments</code> accepted, requestId <code>05a6f0f7-...</code>
06:42:34	Bootstrap VM <code>vcfa-mgmt-5m8gb</code> powered on on esxi04 (2h 17m after submit — already slow)
08:29:48	"Remote deploy the registry on VCF services platform cluster <code>vcf-mgmt-452f36cb72</code> " started
08:58:17	First leader-election timeout in bootstrap VM (logging-operator)
09:02:23–09:02:58	Repeated <code>connection refused to 198.19.0.1:443</code> from CAPV controller
09:03:23	<code>[-]etcd failed: reason withheld</code> in apiserver health check
09:04:12	Fleet attempts VM teardown — fails because VM still powered on
09:12:53	Fleet sets env status to FAILED with <code>LCMVSPHERECONFIG1000095</code>

Total wall time: **4h 47m** before failure.

5.2 Smoking-Gun Log Lines (from `/var/log/vr1cm/vmsp_bootstrap_cd696d6b-...log`)

```
{ "level": "error", "ts": "2026-05-19T08:58:17Z", "msg": "Failed to update lock optimitically: Put \"https://198.19.0.1:443/apis/coordination.k8s.io/v1/namespaces/vmsp-platform/leases/logging-operator.logging.banzaicloud.io\": context deadline exceeded, falling back to slow path" }

{ "level": "info", "ts": "2026-05-19T08:58:17Z", "msg": "failed to renew lease vmsp-platform/logging-operator.logging.banzaicloud.io: timed out waiting for the condition" }

{ "level": "error", "ts": "2026-05-19T08:58:17Z", "logger": "setup", "msg": "problem running manager", "error": "leader election lost" }
```

And the definitive kube-controller-manager error:

```
"Error building controller context" err="failed to wait for apiserver being healthy: timed out waiting for the condition: failed to get apiserver /healthz status:
[+]ping ok
[+]log ok
[-]etcd failed: reason withheld
..."
```

`[-]etcd failed: reason withheld` is the apiserver telling us that the etcd backend has failed an internal health check. apiserver does not surface the underlying etcd error in the healthz response — the actual etcd error is in the etcd pod logs, which by this point are also unavailable because the bootstrap workflow is tearing down.

5.3 Pre-Fix Host Settings (Captured 12:00 UTC)

Host	VSANDeviceMonitoring	lsomSlowDeviceUnmount	SwapThickProvisionDisabled	GuestUnmap	FakeSCSIReservations
esxi01	1	1	0	0	1
esxi02	1	1	0	0	1
esxi03	1	1	0	0	1
esxi04	1	1	0	0	1

Four out of five wrong on every host.

5.4 Post-Fix Settings (Verified 12:18 UTC)

Host	VSANDeviceMonitoring	lsomSlowDeviceUnmount	SwapThickProvisionDisabled	GuestUnmap	FakeSCSIReservations
esxi01	0	0	1	1	1
esxi02	0	0	1	1	1
esxi03	0	0	1	1	1
esxi04	0	0	1	1	1

All five correct on every host.

6. Apply Procedure (Verified)

6.1 SSH to each host and run

```
esxcli system settings advanced set -o /LSOM/VSANDeviceMonitoring -i 0
esxcli system settings advanced set -o /LSOM/lsomSlowDeviceUnmount -i 0
esxcli system settings advanced set -o /VSAN/SwapThickProvisionDisabled -i 1
esxcli system settings advanced set -o /VSAN/GuestUnmap -i 1
esxcli system settings advanced set -o /VSAN/FakeSCSIReservations -i 1
```

No host reboot required. Settings take effect on the next vSAN operation.

6.2 Batch apply via PowerCLI + Posh-SSH

```
$hosts = @(
    @{Name='esxi01'; IP='192.168.1.74'; Pwd='<ESXI01_PW>'},
    @{Name='esxi02'; IP='192.168.1.75'; Pwd='<LAB_PW>'},
    @{Name='esxi03'; IP='192.168.1.76'; Pwd='<LAB_PW>'},
    @{Name='esxi04'; IP='192.168.1.82'; Pwd='<LAB_PW>'}
)
$script = '@'
esxcli system settings advanced set -o /LSOM/VSANDeviceMonitoring -i 0
esxcli system settings advanced set -o /LSOM/lsomSlowDeviceUnmount -i 0
esxcli system settings advanced set -o /VSAN/SwapThickProvisionDisabled -i 1
esxcli system settings advanced set -o /VSAN/GuestUnmap -i 1
esxcli system settings advanced set -o /VSAN/FakeSCSIReservations -i 1
'@
foreach ($h in $hosts) {
    $pw = ConvertTo-SecureString $h.Pwd -AsPlainText -Force
    $cred = New-Object System.Management.Automation.PSCredential('root', $pw)
```

```
$s = New-SSHSession -ComputerName $h.IP -Credential $cred -AcceptKey -ConnectionTimeout 10
Invoke-SSHCommand -SessionId $s.SessionId -Command $script -TimeOut 30 | Out-Null
Remove-SSHSession -SessionId $s.SessionId | Out-Null
}
```

6.3 Make persistent across reboot (already persistent)

ESXi advanced settings written via `esxcli system settings advanced set` persist across host reboots automatically. No further action needed.

7. Verification

Read back the five values on each host:

```
for s in /LSOM/VSANDeviceMonitoring /LSOM/lomSlowDeviceUnmount \
/VSAN/SwapThickProvisionDisabled /VSAN/GuestUnmap \
/VSAN/FakeSCSIReservations; do
val=$(esxcli system settings advanced list -o $s | awk '/Int Value:/ {print $3}')
echo "$s = $val"
done
```

Expected output:

```
/LSOM/VSANDeviceMonitoring = 0
/LSOM/lomSlowDeviceUnmount = 0
/VSAN/SwapThickProvisionDisabled = 1
/VSAN/GuestUnmap = 1
/VSAN/FakeSCSIReservations = 1
```

8. When NOT to Apply These

These tunings are specifically for *nested vSAN in lab environments*. Do not apply them to a production physical-hardware vSAN cluster.

Setting	Why it's safe for nested	Why it's unsafe for production physical
<code>VSANDeviceMonitoring=0</code>	Latency observations are dominated by lower-layer scheduling, not real device health	You lose early-warning of actual failing drives — SMART degradation, firmware bugs, etc.
<code>lomSlowDeviceUnmount=0</code>	Inner-vSAN can't trust outer host latency as a "bad drive" signal	You lose protection from a drive going pathological and dragging the cluster down
<code>FakeSCSIReservations=1</code>	Inner layer can't pass through real reservations; this lets the stack work	On real shared storage with multiple hosts, fake reservations could allow data corruption in MSCS scenarios

For nested labs: apply. For physical production: do NOT apply.

9. Incident Log — 2026-05-19

Time (UTC)	Event
11:45	Investigated env <code>cd696d6b</code> FAILED status — found <code>LCMVSPHERECONFIG1000095</code> with etcd-related root cause in <code>/var/log/vr1cm/vmsp_bootstrap_cd696d6b-...log</code>
11:55	Web search located William Lam article on VCF 9.1 ESX lab workarounds
12:00	Verified all 4 nested ESXi hosts had 4 of 5 settings wrong
12:18	Applied corrected settings on all 4 hosts via <code>esxcli</code> . All five verify correct on all four hosts.
12:21	Snapshotted Fleet (<code>pre-db-cleanup-cd696d6b-2026-05-19</code>)
12:22	Force-deleted orphan bootstrap VM <code>vcfa-mgmt-5m8gb</code> from <code>esxi04</code>
12:23	Postgres Method 4 cleanup of env <code>cd696d6b</code> (env+product+products_nodes+infra_props+exec_orphans, total 114 rows)
12:24	Re-submitted same VCFA API deploy JSON — env <code>aad6f15e-08b1-4548-a192-87d14d493bfb</code> , requestId <code>affe1df5-5152-4cd4-b2bb-29374a0e22cc</code>
12:55+	Monitoring scheduled at +30 min, +60 min, +180 min

10. Lessons Learned

1. Default ESXi settings are tuned for physical hardware. Nested ESXi inherits the same defaults but the optimizations they apply (LSOM device monitoring, slow-device unmount, real SCSI reservations) are actively harmful when the storage layer is a virtual disk on a busy outer host.

2. The William Lam article is authoritative for VCF 9 lab tunings. Broadcom does not document these settings in any KB for VCFA failures — their KB 418755 for `LCMVSPHERECONFIG1000095` points at port 443 connectivity, not at storage. The fix comes from community knowledge.

3. [-]etcd failed: reason withheld in the apiserver healthz is the signature of storage-induced etcd failure. When you see that line in a VCFA bootstrap log, suspect storage latency before you suspect Kubernetes or networking. The fact that other healthz check items (`+ping ok` , `+log ok`) are passing while etcd is failing is the smoking-gun pattern.

4. The April 2026 incident in `VCFA-Automation-Deployment-Troubleshooting-Report.md` Section 11.5 named this exact problem. That doc identified HDD-backed storage as the root cause. After our May NVMe migration we assumed the storage axis was solved — but we didn't apply the host-level tunings. The hardware was fast; the host-level vSAN behavior was still wrong.

5. Settings need NO host reboot to take effect. Apply them anytime before a deploy. They affect subsequent vSAN operations only — in-flight operations are unchanged.

6. Apply ALL FIVE. It is the combination, not any single setting, that creates the safe environment. Skipping `1 somSlowDeviceUnmount=0` (the most operationally dangerous default) while applying the others leaves the cascade-failure window open.

7. Always check `esxcli system settings advanced list -o <setting>` **on every host before a VCFA deploy attempt.** Do not assume previous fixes persisted across host rebuilds (the disk-group rebuild work in May reset some advanced settings).

11. Document Information

Field	Value
Title	Nested vSAN ESXi Tunings for VCF 9 Lab Deployments
Document ID	VC-RUNBOOK-NESTED-VSAN-TUNINGS-20260519
Version	1.0
Issued	May 19, 2026
Author	Virtual Control LLC
Classification	Internal — Lab Environment
Source	William Lam, "VCF 9.1 Comprehensive ESX Configuration Workarounds for Lab Deployments" (williamlam.com, 2026-05)
Related Documents	Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18.md , Nested-ESXi-RAM-Rebalance-for-VCFA-Deploy-2026-05-18.md , VCF-Automation-Deployment-Troubleshooting-Report.md Section 11.5
Distribution	Lab notebook

Revision History

Version	Date	Notes
1.0	May 19, 2026	Initial release. Triggered by env <code>cd696d6b</code> failure with <code>LCMVSPHERECONFIG1000095</code> at 09:12 UTC. Documents the 5 williamlam-prescribed ESXi advanced settings, their effect on nested vSAN behavior under VCFA bootstrap burst writes, the verified pre/post state across all 4 nested hosts (esxi01/02/03/04), the apply procedure (esxcli, no reboot), and the operational guard rails (do NOT apply in physical production). Includes the 9:12 UTC bootstrap log evidence and the etcd-storage-latency cascade explanation. Follow-up retry env is <code>aad6f15e-08b1-4548-a192-87d14d493bfb</code> .

Document End

See also: - `Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18.md` — the Method 4 Postgres cleanup procedure used after this failure - `Nested-ESXi-RAM-Rebalance-for-VCFA-Deploy-2026-05-18.md` — the host RAM rebalance that preceded this finding - `VCF-Automation-Deployment-Troubleshooting-Report.md` Section 11.5 — the April 2026 RCA that identified storage as the failure axis but did not yet name the williamlam tunings